Informe de implementación del proyecto MS-CFLP-CI

Por: Martín José Marrero Quintans

27 de abril de 2026

Índice

1. Introducción **2**

2. Objetivos **2**

3. Estructura del proyecto **2**

4. Descripción de la implementación **3**
4.1. Núcleo del modelo . . . 3
4.2. Heurística greedy . . . 3
4.3. GRASP . . . 3
4.4. Arquitectura general . . . 3
4.5. Solución inicial del GVNS . . . 4
4.6. Perturbación . . . 4
4.7. Mejora local con VND y Reinforcement Learning . . . 4
4.8. Parámetros expuestos . . . 5
4.9. Interfaz de ejecución . . . 5

5. Métricas utilizadas **5**

6. Parámetros probados **6**
6.1. Ajuste inicial sobre wlp01 . . . 6
6.2. Efecto del número de iteraciones . . . 6

7. Resultados obtenidos **6**

8. Discusión **7**

9. Conclusión **7**

A. Uso del programa **7**

1. Introducción

El problema MS-CFLP-CI (Multi-Source Capacitated Facility Location Problem with Customer Incompatibilities) consiste en seleccionar instalaciones y asignar clientes minimizando el coste total, respetando capacidades y restricciones de incompatibilidad. El proyecto desarrollado no se limita a una única heurística, sino que integra el modelo de datos, la construcción greedy, la construcción GRASP, la búsqueda local y el GVNS como método de mejora global.

El objetivo del trabajo es disponer de una cadena completa de resolución: leer instancias, construir soluciones factibles, mejorarlas de forma iterativa y comparar de manera homogénea los resultados de cada estrategia.

2. Objetivos

Los objetivos principales del desarrollo han sido:

- Implementar y documentar todo el flujo del proyecto MS-CFLP-CI.
- Construir soluciones factibles mediante una heurística greedy y una versión GRASP.
- Integrar la mejora local sobre varios vecindarios y reutilizarla tanto en GRASP como en GVNS.
- Diseñar una perturbación por reinserción de clientes entre instalaciones para el GVNS.
- Incorporar una fase VND con selección ϵ-greedy y actualización de valores Q.
- Exponer por línea de comandos los parámetros más relevantes: iteraciones, ϵ, α y semilla.
- Comparar el rendimiento frente a Greedy y GRASP usando distintas métricas.
- Ajustar parámetros para mejorar el rendimiento y documentar los resultados obtenidos.

3. Estructura del proyecto

Las partes más destacables de la estructura del proyecto son:

- `src/main.cc`: punto de entrada, parseo de comandos y ejecución de Greedy, GRASP, GVNS o comparativa.
- `src/lib/instance.h` y `src/instance.cc`: lectura y acceso a los datos de la instancia.
- `src/lib/solution.h` y `src/solution.cc`: representación de la solución, asignaciones, capacidades y costes.
- `src/greedy.cc`: heurística constructiva determinista.
- `src/grasp.cc`: construcción aleatorizada y mejora local.
- `src/gvns.cc`: metaheurística GVNS con perturbación y VND guiado por RL.
- `src/lib/local_search.h`: interfaz común de los vecindarios locales.
- `src/relocation_search.cc`, `src/client_swap_search.cc`, `src/incompatibility_search.cc`, `src/swap_search.cc`: vecindarios utilizados por la mejora local.
- `src/lib/*.h`: cabeceras de interfaz para solvers, instalaciones, clientes, colores, etc.

- `instances/`: instancias de prueba `wlp01.dzn` a `wlp18.dzn`.
- `results/`: resultados de experimentos y ajuste de parámetros.
- `Makefile`: compilación del ejecutable `ms-cflp-ci`.

De forma resumida, la organización del directorio responde a una separación clásica entre interfaz, implementación y datos de entrada.

4. Descripción de la implementación

4.1. Núcleo del modelo

El proyecto se apoya en dos clases fundamentales. `Instance` encapsula los datos de entrada de cada fichero `.dzn`: número de almacenes, clientes, capacidades, costes fijos, costes de transporte e incompatibilidades. `Solution` representa una solución concreta, incluyendo qué instalaciones están abiertas, cuánta demanda se asigna a cada una, la capacidad restante y el coste total.

Este diseño permite que tanto Greedy como GRASP y GVNS trabajen sobre la misma representación interna, evitando duplicar lógica de validación o de cálculo de costes.

4.2. Heurística greedy

La heurística greedy es la base más simple del proyecto y sirve como referencia inicial. Su funcionamiento es determinista:

1. Selecciona un conjunto de instalaciones con margen de holgura.
2. Abre esas instalaciones en la solución.
3. Asigna cada cliente a la instalación abierta que minimiza el coste de suministro, respetando compatibilidad y capacidad.

Aunque su calidad es limitada frente a estrategias más sofisticadas, es rápida y proporciona un punto de partida factible que se usa también como referencia comparativa.

4.3. GRASP

La versión GRASP introduce aleatorización sobre la construcción greedy mediante una Restricted Candidate List (RCL). En lugar de elegir siempre la mejor opción, se selecciona una solución entre las mejores candidatas, lo que genera diversidad.

Tras la construcción, GRASP aplica una mejora local basada en varios vecindarios. Esta fase es compartida conceptualmente con GVNS, aunque en GRASP la estrategia es más clásica y no incluye perturbaciones globales iterativas.

4.4. Arquitectura general

El GVNS implementado sigue la estructura habitual de este tipo de metaheurísticas y reutiliza los mismos componentes de construcción y mejora:

1. Construcción de una solución inicial.
2. Perturbación de la solución actual para escapar de máximos locales.
3. Mejora local mediante VND.
4. Actualización de la mejor solución encontrada.

5. Repetición durante un número fijo de iteraciones.

La clase `GVNS` encapsula los parámetros del algoritmo y mantiene un generador pseudoaleatorio reproducible con la semilla indicada.

4.5. Solución inicial del GVNS

La búsqueda no comienza desde una solución arbitraria. Primero se comparan varias construcciones de tipo GRASP y se conserva la mejor. Además, esa solución inicial se refuerza con una pasada ligera de búsqueda local para mejorar la base sobre la que trabajar.

Esta decisión es importante porque el rendimiento final del GVNS depende mucho de la calidad del punto de arranque.

4.6. Perturbación

La perturbación se realiza mediante reinserciones de demanda entre instalaciones. A nivel operativo:

- Se selecciona una instalación origen abierta que sirva a al menos un cliente.
- Se elige un cliente asignado a dicha instalación.
- Se busca una instalación destino compatible con capacidad libre.
- Si no existe una instalación destino abierta, se intenta abrir una cerrada compatible.

La perturbación no modifica una sola unidad de forma trivial, sino que mueve cantidades transferibles para que la solución resultante sea suficientemente diferente de la anterior.

4.7. Mejora local con VND y Reinforcement Learning

La fase de mejora usa un conjunto de cuatro vecindarios:

- Relocation.
- Client Swap.
- Incompatibility Elimination.
- Facility Swap.

La selección del vecindario sigue una política ϵ-greedy. Con probabilidad ϵ se elige un vecindario aleatorio; en caso contrario se toma el de mayor valor Q entre los activos.

La actualización de los valores Q sigue la regla:

$$Q \leftarrow Q + \alpha(r - Q)$$

donde la recompensa se define de forma proporcional al descenso de coste:

$$r = \frac{f(S) - f(S')}{f(S)}$$

con truncado en el intervalo $[0, 1]$. Esta recompensa resulta adecuada porque distingue mejoras grandes de mejoras pequeñas sin requerir una estimación global del máximo posible.

4.8. Parámetros expuestos

Los parámetros principales del GVNS son:

- `max_iterations`: número de iteraciones del bucle principal.
- `slack_facilities`: margen de instalaciones adicionales abiertas en la construcción inicial.
- `seed`: semilla para reproducibilidad.
- `epsilon`: probabilidad de exploración en la política ϵ-greedy.
- `alpha`: tasa de aprendizaje para la actualización Q.
- `perturbation_strength`: número de reinserciones por iteración.

En la versión final se mantuvo ϵ constante durante la ejecución, ya que el decaimiento automático hacia valores demasiado similares dificultaba distinguir el efecto real de los hiperparámetros.

4.9. Interfaz de ejecución

El fichero `main.cc` actúa como interfaz de usuario del proyecto. Permite ejecutar cada heurística por separado o bien comparar varias en una misma salida:

- `greedy`: construcción rápida y determinista.
- `grasp`: construcción aleatorizada con mejora local.
- `gvns`: fase de perturbación + VND con RL.
- `all`: comparativa conjunta de Greedy, GRASP y GVNS.

Este diseño facilita probar distintas configuraciones sin recompilar y permite medir el impacto real de los cambios sobre una misma instancia.

5. Métricas utilizadas

Para evaluar la calidad de cada solución se han considerado estas métricas:

- **Coste total**: suma del coste fijo de apertura y del coste de transporte.
- **Coste fijo**: coste asociado a las instalaciones abiertas.
- **Coste de transporte**: coste derivado de la asignación de demanda.
- **Instalaciones abiertas**: relación entre instalaciones abiertas y totales.
- **Demanda no servida**: debe ser 0 para aceptar la solución.
- **Incompatibilidad**: debe ser nula; se informa como `NO (valid)` cuando la solución es válida.
- **Error relativo**: porcentaje de desviación respecto al valor óptimo conocido.

La métrica de error relativo es especialmente útil para comparar instancias con escalas diferentes:

$$exterror(\%) = \frac{\text{coste encontrado} - \text{coste óptimo}}{\text{coste óptimo}} \cdot 100$$

6. Parámetros probados

Durante el ajuste se probaron distintas combinaciones de α y ϵ, además de variar el número de iteraciones del GVNS.

6.1. Ajuste inicial sobre wlp01

En una primera fase de ajuste se probaron las siguientes combinaciones sobre `wlp01`:

oprule α	ϵ	Coste total	Error
0.1	0.1	32637	13.65 %
0.2	0.2	32615	13.58 %
0.3	0.1	32637	13.65 %

La configuración $\alpha = 0{,}2$, $\epsilon = 0{,}2$ fue la mejor en ese barrido inicial y se mantuvo como valor base.

6.2. Efecto del número de iteraciones

Se observó que el parámetro más influyente en la calidad final era el número de iteraciones del bucle principal. Con más iteraciones, el algoritmo dedicaba más tiempo a explorar soluciones vecinas y mejoraba de forma clara.

oprule Iteraciones	Coste total	Error	Tiempo aproximado
50	32400	12.83 %	∼30 s
100	32384	12.77 %	∼1 min
200	32108	11.81 %	∼1.5 min
300	32099	11.78 %	∼2 min
500	32001	11.44 %	∼3 min

7. Resultados obtenidos

Se realizaron pruebas en las primeras cuatro instancias, sin embargo, en este informe se denotan únicamente resultados de la primera instancia, meramente por simplificación de la explicación. La comparación sobre `wlp01` entre las tres heurísticas principales produce el siguiente resultado:

oprule Algoritmo	Coste total	Coste fijo	Transporte	Error
Greedy	33422	16980	16442	16.39 %
GRASP	32682	16980	15702	13.81 %
GVNS	32001	16980	15021	11.44 %

GVNS obtiene el mejor coste de los tres, reduciendo el error respecto a Greedy y GRASP. En particular, la mejora frente a GRASP es de más de dos puntos porcentuales, lo que confirma que la combinación de perturbación + VND con RL sí añade capacidad de explotación adicional.

Sin embargo, el error final sigue lejos del objetivo ideal de menos del 5 %. Esto sugiere que la estrategia implementada mejora el punto de partida de manera consistente, pero no es todavía lo bastante agresiva ni sofisticada para acercarse al *optimum* en todas las instancias.

8. Discusión

Los experimentos permitieron extraer varias conclusiones:

- El ajuste de α y ϵ tiene impacto real solo si la búsqueda se ejecuta suficiente tiempo y no se aplana la exploración con mecanismos que hagan converger rápidamente la política.
- Una inicialización excesivamente costosa reduce el tiempo disponible para el bucle principal, empeorando la calidad final.
- La perturbación por reinserción funciona mejor cuando es moderada: demasiado poca no saca al algoritmo del máximo local, y demasiada destruye la estructura útil.
- La recompensa proporcional es una elección razonable porque permite distinguir la magnitud de la mejora.

9. Conclusión

Se ha implementado un GVNS funcional con reinserción como perturbación y VND con Reinforcement Learning como mejora local. La solución final es capaz de superar claramente a Greedy y GRASP sobre las instancias probadas, y el proceso de ajuste ha servido para identificar una configuración más estable y reproducible.

La versión final consigue una mejora apreciable en coste y mantiene validez en las soluciones generadas, aunque no alcanza todavía el umbral de error inferior al 5 %. Para llegar a ese nivel harían falta estrategias más avanzadas de diversificación e intensificación, o un presupuesto de cómputo notablemente mayor.

A. Uso del programa

Ejemplos de ejecución:

```
./ms-cflp-ci instances/wlp01.dzn greedy [slack_k]
./ms-cflp-ci instances/wlp01.dzn grasp [slack_k] [rcl_size] [seed] [iterations]
./ms-cflp-ci instances/wlp01.dzn gvns [slack_k] [max_iterations] [seed] [epsilon] [
    alpha]
./ms-cflp-ci instances/wlp01.dzn all [slack_k] [rcl_size] [seed] [iterations]
```